```
    latents = scheduler.step(noise_pred, t, latents).prev_sample
```

We now use the vae to decode the generated latents back into the image.

```
# scale and decode the image latents with vae
latents = 1 / 0.18215 * latents
with torch.no_grad():
    image = vae.decode(latents).sample
```

And finally, we can now change the image to PIL so we can display or save it.

```
image = (image / 2 + 0.5).clamp(0, 1)
image = image.detach().cpu().permute(0, 2, 3, 1).numpy()
images = (image * 255).round().astype("uint8")
pil_images = [Image.fromarray(image) for image in images]
Pil_images[0]
```

More on how diffusion models work

Diffusion models are a type of generative model that can synthesize high-quality images using a latent variable. Although this may sound similar to what GANs do, there are significant differences between the two approaches, as well as other generative models such as VAEs and flow-based models.

While GANs have been the go-to approach for generating images for several years, especially when working with specific distributions such as human faces or dog breeds, they rely on a different training mechanism. GANs involve two models - a generator and a discriminator - where the generator attempts to produce images that trick the discriminator into thinking they come from the real data distribution. While the idea of two models training each other is intriguing, GANs are notoriously difficult to train, with generators that don't learn or that fall into mode collapse - where they generate the same image repeatedly - being a common issue.

While GANs generate images by training two models to compete with each other, diffusion models generate a sequence of N increasingly noisy